

Guide d'utilisation — Workflow d'architecture de solution avec Multica

Ce guide explique **comment utiliser au quotidien** le workflow d'architecture de solution du dépôt `homelab-portfolio`, orchestré par des agents Multica. Il s'adresse aux humains qui pilotent une demande d'architecture ; il ne remplace pas la source technique du workflow (`core/common/conductor.md` et `core/common/protocols/`), qu'il vulgarise.

1. Ce qu'est ce workflow

Le workflow transforme une **demande d'architecture** (concevoir/faire évoluer une solution, un système, une intégration) en **livrables tracés et validés** : documentation d'architecture, décisions structurantes, diagrammes. Il repose sur trois principes :

- **Orchestration A2A (agent-to-agent)** : un agent **coordinateur** découpe le travail et délègue à des agents spécialisés ; il ne produit pas lui-même les livrables.
- **Validation humaine granulaire** : vous validez **chaque choix séparément** — rien n'avance sans votre accord.
- **Tout est tracé sur l'issue Multica** : analyses, décisions, délégations, contrôles sécurité et validations vivent dans les commentaires de l'issue (piste d'audit unique).

Le workflow **s'adapte au travail** : une demande simple reçoit un traitement léger, une demande complexe le traitement complet. Ce réglage est outillé par les **scopes** et deux axes indépendants (**profondeur** des artefacts et **niveau de vérification** des livrables) — voir `core/common/protocols/scopes-and-axes.md`.

2. Choisir le bon workflow (règle de routage)

Le dépôt porte **deux workflows totalement indépendants**. Avant tout, la demande est classée dans **l'un ou l'autre** — jamais les deux :

La demande porte sur...	Workflow	Coordinateur
Architecture de solution : documentation (DAS), décisions, diagrammes, choix techno, intégration, cybersécurité, AWS, cycle spec-driven (OpenSpec)	Architecture (ce guide)	Architecture Solution & Intégration
Homelab : stack Docker/Proxmox, <code>docker-compose</code> , Terraform de stack, n8n, Home Assistant, Vault, Traefik	Homelab	Tech Lead

En cas de doute, le coordinateur **vous demande de trancher** avant d'engager quoi que ce soit.

3. Comment démarrer une demande

1. **Créer une issue Multica** décrivant l'objectif (le *pourquoi* et le *quoi*), et l'**assigner au coordinateur** (agent « Architecture Solution & Intégration ») — ou le mentionner sur une issue existante.
2. Le coordinateur **charge le contexte** (répertoire du projet, documentation et décisions existantes) de façon optimisée, puis confirme le périmètre.
3. Il vous présente ses propositions **choix par choix** aux points de validation (voir §5).
4. Toute la conversation reste **sur l'issue** : c'est là que vous suivez l'avancement et rendez vos décisions.

Rien à impact ne se fait sans votre validation explicite, et aucune action destructive sans plan de rollback validé.

4. Les 5 phases

Le cycle est structuré en cinq phases (nomenclature AI-DLC adaptée au workspace) :

#	Phase	Ce qui s'y passe	Gate humain
0	Initialization	Bootstrap déterministe : vérification du répertoire projet, détection brownfield/greenfield, initialisation de la piste d'audit	Non (automatique)
1	Ideation	Capture d'intention, faisabilité/contraintes, définition du périmètre, (maquettes si UI), approbation	Approbation intention + périmètre
2	Inception	Cadrage, chargement du contexte existant, analyse des besoins, découpage des livrables, conception + décisions structurantes	Validation granulaire
3	Construction	« Walking skeleton » (première tranche de bout en bout), production détaillée, contrôle sécurité + cohérence, consolidation	Validation granulaire
4	Operation	Déploiement sous validation, notification de fin, maintenance/support	Validation humaine explicite

Chaque phase se décompose en **stages** (une fiche par stage dans `core/common/stages/<phase>/`).

5. Comment vous validez : la boucle Keep / Modify / Redo

À chaque point de validation, le coordinateur présente **chaque élément séparément** (le choix, sa justification, l'alternative). Pour chacun, vous répondez :

- ✓ **Keep** — l'élément est validé, on avance.
- ☐ **Modify** — vous reformulez ; le coordinateur ajuste **cet élément** et le re-présente.
- ✗ **Redo** — vous rejetez ; le coordinateur propose une alternative pour **cet élément**.

Le coordinateur **ne fusionne jamais** les choix en un « tout ou rien », et n'avance jamais sur un élément non validé.

6. Qui fait quoi (rôles A2A)

Les agents sont désignés par leur **fonction** :

Fonction	Rôle dans le workflow
Architecture Solution & Intégration	Coordinateur : découpe, délègue, contrôle, sollicite la sécurité, demande vos validations
Architecte de solution	Produit la documentation d'architecture, les décisions, les diagrammes
Architecte AWS	Conçoit l'architecture AWS, chiffre et optimise les coûts
Infrastructure Windows	Migration/administration Windows, déploiement sous validation
OpenSpec Expert	Cycle spec-driven OpenSpec (si activé)
Architecte Cybersécurité	Analyse de sécurité (OWASP, STRIDE... ; normes avancées sur demande)
Experte d'archivage	Mise à disposition/versionnement des livrables
Agent de notifications	Notification de fin de tâche (ntfy), sur demande
Vente & Appels d'Offres	Synthèse des architectures en supports commerciaux

La délégation se fait par **mention** sur l'issue ; l'agent sollicité répond sur la même issue.

7. Fiabilisation automatique (sans alourdir votre charge)

- **Vérification gates** : à chaque frontière de phase, un contrôle **automatique** de traçabilité (artefacts présents, exigences reliées à une décision/livrable, pas d'orphelin) poste un « Rapport de vérification » sur l'issue.
- **Sensors** : checks déterministes déclenchés à l'écriture d'un artefact (ex. sections requises, couverture amont).

Ces mécanismes sont **advisory** : ils factuelisent la qualité mais **ne bloquent jamais** votre validation ni le contrôle sécurité. Détail : `core/sensors/` et `core/common/protocols/`.

8. Sécurité et décisions

- **Contrôle sécurité systématique** : toute modification d'architecture passe par l'Architecte cybersécurité. OWASP + STRIDE sont toujours actifs ; les normes avancées (PCI DSS, GDPR, Loi 25, LPRPDE) ne s'appliquent **que si vous les demandez explicitement**.
- **Décisions structurantes** : chaque décision importante est tracée dans le **registre de décisions** (`decisions/`), revue et approuvée par vous.
- **Boucle d'apprentissage** : vos corrections récurrentes peuvent être capitalisées en **règles persistantes** (`core/rules/`), appliquées au **prochain** workflow (jamais en cours de route), toujours après votre validation.

9. Agents et skills côté Multica (important)

Le dépôt est la **source** ; l'exécution vit dans le workspace Multica :

- Les **définitions d'agents** sont versionnées dans `core/agents/` (copie de référence). Les agents actifs sont ceux du workspace.
- Les **skills** vivent dans `plugins/<nom>/skills/` (format Agent Plugins v1.0.0). **Un `SKILL.md` du dépôt n'est pas utilisé automatiquement** : sur Multica, une skill doit être **importée** (`multica skill import`) puis **assignée** à un agent (`multica agent skills add`).
- Les agents chargent leurs skills **au démarrage de session** et le reste **à la demande** (chargement optimisé).

10. En résumé — le parcours type

1. Vous **créez l'issue** et l'assignez au coordinateur.
2. **Ideation** : le coordinateur cadre l'intention et le périmètre → vous approuvez.
3. **Inception** : besoins, conception, décisions → vous validez choix par choix (Keep/Modify/Redo), avec contrôle sécurité.
4. **Construction** : walking skeleton puis production détaillée → validations granulaires + sécurité.
5. **Operation** : déploiement **sous votre validation explicite**, puis notification et mise à disposition.
6. À tout moment : **l'issue** est votre tableau de bord (audit, décisions, livrables).

Sources techniques : `core/common/conductor.md` (instructions du coordinateur), `core/common/stages/` (fiches de stage), `core/common/protocols/` (gouvernance & sécurité, reviewer, scopes & axes, définition/protocole de stage), `core/rules/`, `core/sensors/`, `decisions/`. Règle de routage : `AGENTS.md`.
